

CSS Next Steps

CSS Variables & Responsive Design — Made Simple

2-Hour Plan | Easy Step-by-Step Notes | Practice Questions Included

Hour 1 — CSS Variables (Custom Properties)

Step 1: What problem are we solving?

Say you used the color blue in 10 places in your CSS — buttons, headings, borders. Now you want to change it to green.

Without variables: you go and change it 10 times, by hand. Miss one spot, and your design looks broken.

Simple fix: Give the color a name ONE time. Use that name everywhere instead of the actual color. Now if you want to change the color, you only change it in ONE place — everywhere it's used updates by itself.

Step 2: How to create a variable

We write our variables inside something called `:root`. Just think of `:root` as "the whole page." If you put a variable here, it can be used ANYWHERE on your page.

```
:root {  
  --primary-color: #3498db;  
  --spacing: 16px;  
  --font-main: 'Poppins', sans-serif;  
}
```

Line 1: `:root` — this is just where we keep all our variables.

Line 2: Every variable name starts with two dashes `--`. That's just the rule. It tells CSS "this is a variable."

Line 3 & 4: You can store colors, sizes, fonts — anything — inside a variable.

Step 3: How to USE the variable

To use it, just wrap the variable name inside `var()`, like this:

```
.button {  
  background-color: var(--primary-color);  
  padding: var(--spacing);  
  font-family: var(--font-main);  
}
```

What's really happening: The browser sees `var(--primary-color)` and simply swaps it out for the color we saved earlier — `#3498db`. Change that ONE saved value, and every place using it changes too.

Step 4: A backup value, just in case

What if the variable name has a typo, or doesn't exist? You can give `var()` a second value to use as a backup.

```
color: var(--text-color, black);
```

If `--text-color` doesn't exist, the browser just uses **black** instead. Nothing breaks.

Step 5: Changing a variable for just ONE section

You can also give the SAME variable a different value, but only for one specific part of the page.

```
.dark-theme {  
  --primary-color: #1abc9c;  
}
```

Simple example: Anything inside `.dark-theme` now gets the new color `#1abc9c`. Everything OUTSIDE `.dark-theme` still uses the old color. This is basically how "dark mode" works on real apps and websites.

Step 6: A quick comparison (just to know the difference)

SASS Variables

CSS Variables

Compiled once, then frozen (static)	Live — can change anytime, even after page loads
Cannot be changed with JavaScript	Can be changed with JavaScript in real-time
Needs a build tool (SASS compiler)	Works directly in the browser, no tools needed

Step 7: Try it with your own eyes

Make a button whose color comes from a variable. Then paste this in your browser console and press Enter:

```
// paste this in browser console
document.documentElement.style.setProperty('--primary-color', 'red');
```

The button's color changes instantly — and you never touched your CSS file. That's what makes CSS variables special.

Hour 2 — Responsive Design & Media Queries

Step 1: What problem are we solving?

Your website might be opened on a phone, a tablet, or a big computer screen. A layout that looks perfect on a laptop can look completely squished and broken on a phone.

Simple fix: We write special CSS rules that only "switch on" when the screen is a certain size. This is called a Media Query.

Step 2: The Basic Syntax

```
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

@media — means "only use the CSS inside this box if a condition is true."

(max-width: 768px) — the condition. In plain words: "only if the screen is 768px wide or smaller."

Inside the curly braces — just normal CSS, but it only turns on when that condition is true.

Step 3: Common screen sizes people use

```
/* Mobile phones */  
@media (max-width: 480px) { }  
  
/* Tablets */  
@media (max-width: 768px) { }  
  
/* Small laptops */  
@media (max-width: 1024px) { }
```

These numbers are not a strict rule — just common sizes most developers agree on. You don't need to memorize them, just remember roughly: small = phone, medium = tablet, large = laptop/desktop.

Step 4: Start small, then go big (Mobile-first)

Most developers design for phones FIRST, and then adjust for bigger screens. This is easier than starting big and trying to shrink everything down later.

```
.card {  
  width: 100%; /* default: full width on small screens */  
}  
  
@media (min-width: 768px) {  
  .card {  
    width: 50%; /* becomes half-width on bigger screens */  
  }  
}
```

Why start small? More people use phones than computers these days. It's simpler to start small and grow, than to start big and squeeze things down.

Step 5: One line you must NOT forget

Put this one line inside your HTML's <head> section. Without it, your media queries won't work properly on real phones.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

In simple words: "Match the page width to the phone's actual screen width, and don't zoom in or out automatically." Every responsive website has this line.

Step 6: Putting both hours together

Now let's connect what we learned. We can change a VARIABLE'S value inside a media query — and everything using that variable changes automatically too.

```
:root {  
  --font-size: 16px;  
}  
  
@media (max-width: 480px) {  
  :root {  
    --font-size: 14px;  
  }  
}
```

On a phone screen (480px or smaller), `--font-size` becomes 14px instead of 16px — and every single element using it shrinks together, automatically. You didn't have to change font-size on each element one by one.

Practice Question Set

Part A Conceptual (Short Answer)

1. What's the difference between a CSS variable and a SASS/SCSS variable?
2. Why do we usually declare variables inside `:root`?
3. What does `max-width: 768px` in a media query actually mean — does the style apply **above** or **below** that width?
4. Why is "mobile-first" design generally recommended over "desktop-first"?
5. What happens if you use `var(--x)` but `--x` was never defined and no fallback is given?

Part B Code Output Prediction

```
:root {  
  --color: blue;  
}  
.box {  
  color: var(--color);  
}  
.dark .box {  
  --color: white;  
}
```

Q6. If `.box` is placed inside an element with class `.dark`, what color will the text be? Why?

Part C Practical Coding Tasks

7. Create a `:root` block with variables for `--primary-color`, `--secondary-color`, and `--padding`. Style a card using **ONLY** these variables.
8. Write a media query that turns a 3-column flex layout into a single column when the screen is below 600px wide.
9. Build a navbar that shows full menu items on desktop but shows just the text "Menu" below 768px (no JS toggle logic needed yet).
10. Combine both concepts: create a heading whose `font-size` is a variable, and reduce that variable's value inside a media query for small screens.

Keep practicing — try writing the code for each question yourself in a code editor before checking answers. That's how it truly sticks.